



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Інженерія програмного забезпечення

Практична робота №6

«Породжувальні шаблони. Шаблони Prototype, Singleton, Factory Method, Abstract
Factory, Builder»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Васильєва М. Д.*

Тема: «Породжувальні шаблони. Шаблони Prototype, Singleton, Factory Method, Abstract Factory, Builder».

Мета: Вивчення шаблонів поведінки. Отримання базових навичок з застосування шаблонів Prototype, Singleton, Factory Method Abstract Factory та Builder.

Практична частина (Завдання №1)

Мій номер залікової книги: 4106, звідси $4106 \bmod 7 = 4$. Тому умова до 1 завдання буде:

Необхідно створити модель для подання розпорядника гри (**GameManager**), який складається з ігрового простору та списку ігрових фішок. Потрібно забезпечити можливість створення **тільки одного примірника** розпорядника за допомогою шаблону *Singleton*.

Вимоги

- **Клас розпорядника гри (GameManager):**
 - має бути єдиною точкою керування грою;
 - повинен зберігати стан ігрового простору (наприклад, поле, карта, рівень);
 - повинен містити список ігрових фішок (об'єкти, які представляють гравців або елементи гри);
 - має забезпечувати методи для додавання, видалення та оновлення фішок;
 - має містити метод для запуску або перезапуску гри.
- **Гарантія єдиного примірника:**
 - якщо інші частини програми спробують створити новий об'єкт **GameManager**, це повинно бути неможливо;
 - усі звернення до розпорядника гри мають здійснюватися лише через уже існуючий примірник.
- **Допоміжні класи (за бажанням для зручності):**
 - **GameSpace** — описує ігрове поле або середовище;
 - **GamePiece** — представляє окрему ігрову фішку (позиція, тип, стан).
- **Загальна структура системи:**
 - система повинна бути спроектована так, щоб усі елементи гри взаємодіяли між собою **через GameManager**.

У методі main продемонструвати:

- створення кількох «об'єктів» **GameManager** і показати, що в усіх випадках повертається один і той самий екземпляр (робота шаблону *Singleton*);
- додавання кількох ігрових фішок у спільний ігровий простір через **GameManager**;
- виведення поточного стану гри (наприклад, параметри простору та список фішок);
- підтвердити, що зміни, внесені через один доступ до **GameManager**, відображаються при зверненні до нього з будь-якої іншої частини програми.

Загальна структура проекту до компіляції:

```
1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task1> tree /f /a
2 Folder PATH listing
3 Volume serial number is 1A5B-F8AF
4 C:.
5 |   build.xml
6 |
7 \---src
8     \---work5
9         GameManager.java
10        GameManagerDemo.java
11        GamePiece.java
12        GameSpace.java
```

Код build.xml задля автоматизації:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project name="task3" default="run" basedir=".">
3
4     <!-- Основні директорії проекту -->
5     <property name="src.dir" value="src"/>
6     <property name="build.dir" value="build"/>
7     <property name="dist.dir" value="dist"/>
8     <property name="doc.dir" value="doc"/>
9
10    <!-- Ініціалізація: створення службових папок -->
11    <target name="init" description="Create build, dist and doc directories">
12        <mkdir dir="${build.dir}"/>
13        <mkdir dir="${dist.dir}"/>
14        <mkdir dir="${doc.dir}"/>
15    </target>
16
17    <!-- Компіляція вихідних кодів -->
18    <target name="compile" depends="init" description="Compile source code">
19        <javac srcdir="${src.dir}" destdir="${build.dir}" includeantruntime="false"
   ↪     encoding="UTF-8"/>
20    </target>
21
22    <!-- Генерація Javadoc (документація українською) -->
23    <target name="javadoc" depends="compile" description="Generate Javadoc
   ↪     documentation">
24        <javadoc destdir="${doc.dir}"
25            sourcepath="${src.dir}"
26            packagenames="work5"
27            encoding="UTF-8"
28            charset="UTF-8"
29            docencoding="UTF-8"
30            windowtitle="task3 API"
31            doctitle="task3 API">
32        </javadoc>
33    </target>
34
```

```

35     <!-- Створення jar-файлу -->
36     <target name="jar" depends="compile" description="Create JAR archive">
37         <jar destfile="${dist.dir}/task3.jar" basedir="${build.dir}">
38             <manifest>
39                 <attribute name="Main-Class" value="work5.GameManagerDemo"/>
40             </manifest>
41         </jar>
42     </target>
43
44     <!-- Запуск демо-програми -->
45     <target name="run" depends="compile,javadoc" description="Run demo program">
46         <java classname="work5.GameManagerDemo" fork="true">
47             <classpath>
48                 <pathelement path="${build.dir}"/>
49             </classpath>
50         </java>
51     </target>
52
53     <!-- Прибирання результатів збірки -->
54     <target name="clean" description="Delete build, dist and doc directories">
55         <delete dir="${build.dir}"/>
56         <delete dir="${dist.dir}"/>
57         <delete dir="${doc.dir}"/>
58     </target>
59
60 </project>

```

Демонстрація доступних команд збірки:

```

1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task1> ant -p
2 Buildfile: C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task1\build.xml
3
4 Main targets:
5
6 clean    Delete build, dist and doc directories
7 compile  Compile source code
8 init     Create build, dist and doc directories
9 jar      Create JAR archive
10 javadoc  Generate Javadoc documentation
11 run      Run demo program
12 Default target: run

```

Вихідний код *.java буде на віддаленому репозиторії, посилання на яке буде додане разом із звітом.

Частина виводу результату збірки демонстрації виконання програми:

```

1 ...
2
3 run:
4     [java] Checking Singleton:

```

```

5      [java] manager1 == manager2 -> true
6      [java]
7      [java] Initial game state (via manager1):
8      [java] === Game state ===
9      [java] Game number: 1
10     [java] Game space: GameSpace{name='Dungeon level', width=20, height=15}
11     [java] Pieces:
12     [java]   GamePiece{id=1, type='Hero', x=5, y=5}
13     [java]   GamePiece{id=2, type='Monster', x=10, y=7}
14     [java]   GamePiece{id=3, type='Chest', x=3, y=2}
15     [java] =====
16     [java]
17     [java] Moving piece with id = 2 (Monster) to (12, 8)...
18     [java] Game state after movement (via manager2):
19     [java] === Game state ===
20     [java] Game number: 1
21     [java] Game space: GameSpace{name='Dungeon level', width=20, height=15}
22     [java] Pieces:
23     [java]   GamePiece{id=1, type='Hero', x=5, y=5}
24     [java]   GamePiece{id=2, type='Monster', x=12, y=8}
25     [java]   GamePiece{id=3, type='Chest', x=3, y=2}
26     [java] =====
27     [java]
28     [java] Removing piece with id = 3 (Chest)...
29     [java] Game state after removing chest (via manager1):
30     [java] === Game state ===
31     [java] Game number: 1
32     [java] Game space: GameSpace{name='Dungeon level', width=20, height=15}
33     [java] Pieces:
34     [java]   GamePiece{id=1, type='Hero', x=5, y=5}
35     [java]   GamePiece{id=2, type='Monster', x=12, y=8}
36     [java] =====
37     [java]
38     [java] Restarting game...
39     [java] Game state after restart (via manager2):
40     [java] === Game state ===
41     [java] Game number: 2
42     [java] Game space: GameSpace{name='Dungeon level', width=20, height=15}
43     [java] Pieces:
44     [java]   (no pieces)
45     [java] =====
46
47 BUILD SUCCESSFUL
48 Total time: 2 seconds

```

Загальна структура проекту після виконання збірки:

```

1  PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task1> tree /f /a
2  Folder PATH listing
3  Volume serial number is 1A5B-F8AF
4  C:..
5  |   build.xml
6  |

```

```

7  +---build
8  |   \---work5
9  |           GameManager.class
10 |           GameManagerDemo.class
11 |           GamePiece.class
12 |           GameSpace.class
13 |
14 +---dist
15 +---doc
16 | |   allclasses-index.html
17 | |   allpackages-index.html
18 | |   copy.svg
19 | |   element-list
20 | |   help-doc.html
21 | |   index-all.html
22 | |   index.html
23 | |   link.svg
24 | |   member-search-index.js
25 | |   module-search-index.js
26 | |   overview-tree.html
27 | |   package-search-index.js
28 | |   script.js
29 | |   search-page.js
30 | |   search.html
31 | |   search.js
32 | |   stylesheet.css
33 | |   tag-search-index.js
34 | |   type-search-index.js
35 |
36 | +---legal
37 | |   COPYRIGHT
38 | |   jquery.md
39 | |   jqueryUI.md
40 | |   LICENSE
41 |
42 | +---resources
43 | |   glass.png
44 | |   x.png
45 |
46 | +---script-dir
47 | |   jquery-3.7.1.min.js
48 | |   jquery-ui.min.css
49 | |   jquery-ui.min.js
50 |
51 | \---work5
52 |           GameManager.html
53 |           GameManagerDemo.html
54 |           GamePiece.html
55 |           GameSpace.html
56 |           package-summary.html
57 |           package-tree.html
58 |
59 \---src
60   \---work5

```

61 GameManager.java

62 GameManagerDemo.java

63 GamePiece.java

64 GameSpace.java

Скріншот наявності документації:

Package work5

package work5

Classes

Class	Description
GameManager	Singleton-клас розпорядника гри Містить інформацію про
GameManagerDemo	Демонстраційний клас для перевірки роботи шаблону Sin
GamePiece	Клас, що описує окрему ігрову фішку
GameSpace	Опис ігрового простору (поле, карта, рівень і ь.д.)

Рис. 1: Скріншот документації Javadoc для 1 завдання

UML-діаграма проекту:

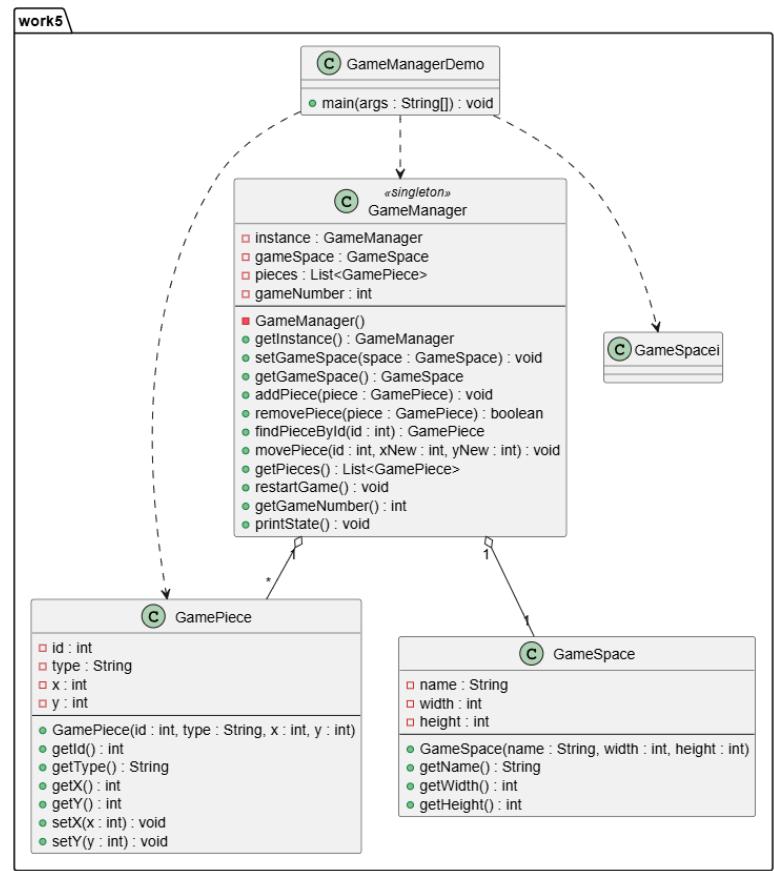


Рис. 2: UML-діаграма проекту для 1 завдання

Практична частина (Завдання №2)

Мій номер залікової книги: 4106, звідси $4106 \bmod 7 = 4$. Тому умова до 2 завдання буде: Необхідно визначити специфікації класів для подання сімейства інструментів універсального інтегрованого середовища розробки (`Validator`, `Compiler`, `Debugger`) з їх реалізацією для різних мов (`Java`, `C++`).

Вимоги

- **Фабрика інструментів середовища розробки:**
 - створити абстрактний клас або інтерфейс `IDEToolsFactory`, який визначає фабричні методи для створення компонентів середовища розробки:
`createValidator()` – повертає об’єкт для перевірки коду, `createCompiler()` – повертає об’єкт компілятора, `createDebugger()` – повертає об’єкт дебагера;
 - класи `JavaIDEToolsFactory` та `CppIDEToolsFactory` є конкретними реалізаціями `IDEToolsFactory`, що створюють інструменти для відповідних мов програмування.
- **Інтерфейси інструментів:**
 - створити абстрактні інтерфейси `Validator`, `Compiler` та `Debugger`, які визначають базові методи роботи, наприклад: `validateCode(String code)` для `Validator`, `compile(String source)` для `Compiler`, `debug(String program)` для `Debugger`.
- **Реалізації для мов програмування:**
 - класи `JavaValidator`, `JavaCompiler`, `JavaDebugger` – реалізації інтерфейсів для мови `Java`;
 - класи `CppValidator`, `CppCompiler`, `CppDebugger` – реалізації інтерфейсів для мови `C++`.
- **Розширюваність системи:**
 - забезпечити можливість прозорого для клієнта розширення системи новими реалізаціями для інших мов без зміни клієнтського коду.

У методі `main` продемонструвати:

- створення фабрики певного типу (наприклад, `JavaIDEToolsFactory` або `CppIDEToolsFactory`);
- за допомогою фабрики створити об’єкти всіх компонентів: `Validator`, `Compiler` і `Debugger`;
- виклик відповідних методів для перевірки, компіляції та відладки умовного фрагмента коду.

Загальна структура проекту до компіляції:

```
1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task2> tree /f /a
2 Folder PATH listing
3 Volume serial number is 1A5B-F8AF
```



```

4 C:.
5 |   build.xml
6 |
7 \---src
8     \---work5
9         Compiler.java
10        CppCompiler.java
11        CppDebugger.java
12        CppIDEToolsFactory.java
13        CppValidator.java
14        Debugger.java
15        IDEToolsDemo.java
16        IDEToolsFactory.java
17        JavaCompiler.java
18        JavaDebugger.java
19        JavaIDEToolsFactory.java
20        JavaValidator.java
21        Validator.java

```

Код build.xml задля автоматизації:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project name="task2" default="run" basedir=".">
3
4     <!-- Основні директорії проекту -->
5     <property name="src.dir" value="src"/>
6     <property name="build.dir" value="build"/>
7     <property name="dist.dir" value="dist"/>
8     <property name="doc.dir" value="doc"/>
9
10    <!-- Ініціалізація: створення службових папок -->
11    <target name="init" description="Create build, dist and doc directories">
12        <mkdir dir="${build.dir}"/>
13        <mkdir dir="${dist.dir}"/>
14        <mkdir dir="${doc.dir}"/>
15    </target>
16
17    <!-- Компіляція вихідних кодів -->
18    <target name="compile" depends="init" description="Compile source code">
19        <javac srcdir="${src.dir}"
20            destdir="${build.dir}"
21            includeantruntime="false"
22            encoding="UTF-8"/>
23    </target>
24
25    <!-- Генерація Javadoc (українська документація) -->
26    <target name="javadoc" depends="compile" description="Generate Javadoc
27        ↪ documentation">
28        <javadoc destdir="${doc.dir}"
29            sourcepath="${src.dir}"
30            packagenames="work5"
31            encoding="UTF-8"
32            charset="UTF-8"
33            docencoding="UTF-8"

```

```

33         windowtitle="IDE Tools Factory API"
34         doctitle="IDE Tools Factory API">
35     </javadoc>
36 </target>
37
38 <!-- Створення jar-файлу -->
39 <target name="jar" depends="compile" description="Create JAR archive">
40     <jar destfile="${dist.dir}/ide-tools-factory.jar" basedir="${build.dir}">
41         <manifest>
42             <attribute name="Main-Class" value="work5.IDEToolsDemo"/>
43         </manifest>
44     </jar>
45 </target>
46
47 <!-- Запуск демо-програми -->
48 <target name="run" depends="compile,javadoc" description="Run demo program">
49     <java classname="work5.IDEToolsDemo" fork="true">
50         <classpath>
51             <pathelement path="${build.dir}"/>
52         </classpath>
53     </java>
54 </target>
55
56 <!-- Очищення результатів збірки -->
57 <target name="clean" description="Delete build, dist and doc directories">
58     <delete dir="${build.dir}"/>
59     <delete dir="${dist.dir}"/>
60     <delete dir="${doc.dir}"/>
61 </target>
62
63 </project>

```

Демонстрація доступних команд збірки:

```

1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task2> ant -p
2 Buildfile: C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task2\build.xml
3
4 Main targets:
5
6 clean    Delete build, dist and doc directories
7 compile  Compile source code
8 init     Create build, dist and doc directories
9 jar      Create JAR archive
10 javadoc  Generate Javadoc documentation
11 run      Run demo program
12 Default target: run

```

Вихідний код *.java буде на віддаленому репозиторії, посилання на яке буде додане разом із звітом.

Частина виводу результату збірки демонстрації виконання програми:

```

1 run:
2     [java] Using Java IDE tools factory:
3     [java] JavaValidator: validating Java code...
4     [java] Code: System.out.println("Hello, Java!");
5     [java] Result: no syntax errors found.
6     [java] JavaCompiler: compiling Java source...
7     [java] Source: System.out.println("Hello, Java!");
8     [java] Result: compilation successful, .class file generated.
9     [java] JavaDebugger: starting Java debug session...
10    [java] Program: JavaSampleProgram
11    [java] Result: breakpoint hit at line 42.
12    [java]
13    [java] Using C++ IDE tools factory:
14    [java] CppValidator: validating C++ code...
15    [java] Code: std::cout << "Hello, C++!" << std::endl;
16    [java] Result: no syntax errors found.
17    [java] CppCompiler: compiling C++ source...
18    [java] Source: std::cout << "Hello, C++!" << std::endl;
19    [java] Result: compilation successful, executable generated.
20    [java] CppDebugger: starting C++ debug session...
21    [java] Program: CppSampleProgram
22    [java] Result: watchpoint triggered on variable x.
23
24 BUILD SUCCESSFUL
25 Total time: 1 second

```

Загальна структура проекту після виконання збірки:

```

1 PS C:\Users\artem\Desktop\reps\university_roadmap\semestr3\software
   ↪ engineering\labs\lab6\task2> tree /f /a
2 Folder PATH listing
3 Volume serial number is 1A5B-F8AF
4 C:..
5 |   build.xml
6 |
7 +---build
8 |   \---work5
9 |           Compiler.class
10 |           CppCompiler.class
11 |           CppDebugger.class
12 |           CppIDEToolsFactory.class
13 |           CppValidator.class
14 |           Debugger.class
15 |           IDEToolsDemo.class
16 |           IDEToolsFactory.class
17 |           JavaCompiler.class
18 |           JavaDebugger.class
19 |           JavaIDEToolsFactory.class
20 |           JavaValidator.class
21 |           Validator.class
22 |
23 +---dist
24 +---doc

```

```

25 | | allclasses-index.html
26 | | allpackages-index.html
27 | | copy.svg
28 | | element-list
29 | | help-doc.html
30 | | index-all.html
31 | | index.html
32 | | link.svg
33 | | member-search-index.js
34 | | module-search-index.js
35 | | overview-tree.html
36 | | package-search-index.js
37 | | script.js
38 | | search-page.js
39 | | search.html
40 | | search.js
41 | | stylesheet.css
42 | | tag-search-index.js
43 | | type-search-index.js
44 | |
45 | +---legal
46 | |     COPYRIGHT
47 | |     jquery.md
48 | |     jqueryUI.md
49 | |     LICENSE
50 | |
51 | +---resources
52 | |     glass.png
53 | |     x.png
54 | |
55 | +---script-dir
56 | |     jquery-3.7.1.min.js
57 | |     jquery-ui.min.css
58 | |     jquery-ui.min.js
59 | |
60 | \---work5
61 |     Compiler.html
62 |     CppCompiler.html
63 |     CppDebugger.html
64 |     CppIDEToolsFactory.html
65 |     CppValidator.html
66 |     Debugger.html
67 |     IDEToolsDemo.html
68 |     IDEToolsFactory.html
69 |     JavaCompiler.html
70 |     JavaDebugger.html
71 |     JavaIDEToolsFactory.html
72 |     JavaValidator.html
73 |     package-summary.html
74 |     package-tree.html
75 |     Validator.html
76 |
77 | \---src
78 |     \---work5

```

```

79     Compiler.java
80     CppCompiler.java
81     CppDebugger.java
82     CppIDEToolsFactory.java
83     CppValidator.java
84     Debugger.java
85     IDEToolsDemo.java
86     IDEToolsFactory.java
87     JavaCompiler.java
88     JavaDebugger.java
89     JavaIDEToolsFactory.java
90     JavaValidator.java
91     Validator.java

```

Скріншот наявності документації:

Package work5

package work5

All Classes and Interfaces	Interfaces	Classes
Class	Description	
Compiler	Інтерфейс компілятора вихідного коду	
CppCompiler	Реалізація компілятора для мови C++	
CppDebugger	Реалізація налагоджувача для мови C++	
CppIDEToolsFactory	Конкретна фабрика інструментів середовища розробки для мови C++	
CppValidator	Реалізація валідатора для мови C++	
Debugger	Інтерфейс налагоджувача програми	
IDEToolsDemo	Демонстраційний клас для перевірки роботи абстрактної фабрики інструментів	
IDEToolsFactory	Абстрактна фабрика інструментів інтегрованого середовища розробки	
JavaCompiler	Реалізація компілятора для мови Java	
JavaDebugger	Реалізація налагоджувача для мови Java	

Рис. 3: Скріншот документації Javadoc для 2 завдання

UML-діаграма проекту:

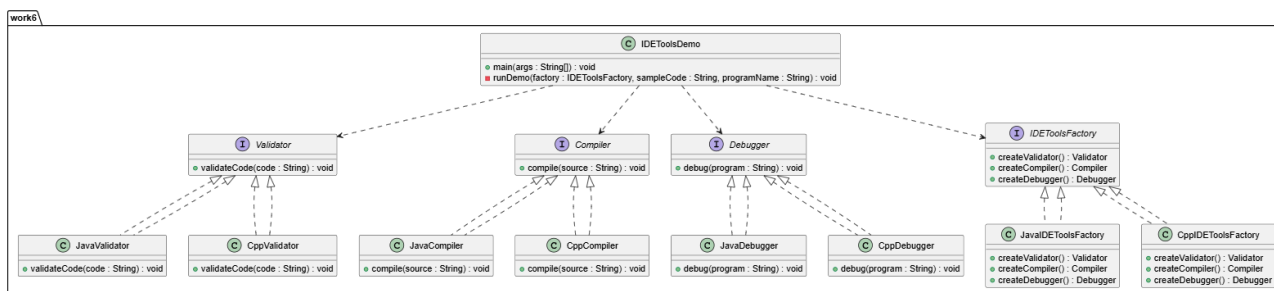


Рис. 4: UML-діаграма проекту для 2 завдання

Висновок: У ході виконання роботи було спроектовано та реалізовано модель сімейства інструментів інтегрованого середовища розробки (IDE), що включає компоненти *Validator*, *Compiler* та *Debugger* для двох мов програмування — Java та C++. Для створення взаємопов'язаних об'єктів цих типів було застосовано шаблон проектування *Abstract Factory* у вигляді інтерфейсу *IDEToolsFactory* та його конкретних реалізацій *JavaIDEToolsFactory* і *CppIDEToolsFactory*. Клієнтський код (*IDEToolsDemo*) працює виключно з абстракціями фабрики та інтерфейсами інструментів, що демонструє ослаблене зв'язування між використанням інструментів і їх конкретними реалізаціями.

Результати запуску програми показали коректність роботи фабрик: для кожного типу фабрики створюється узгоджений набір інструментів (валідатор, компілятор, налагоджувач), над якими послідовно виконуються операції перевірки коду, компіляції та налагодження. Завдяки такій організації системи стає можливим прозоре розширення: для додавання підтримки нової мови достатньо реалізувати нові класи-продукти (*XxxValidator*, *XxxCompiler*, *XxxDebugger*) та відповідну фабрику *XxxIDEToolsFactory*, не змінюючи клієнтський код.

Таким чином, мету роботи досягнуто: розроблено узагальнену структуру IDE-інструментів із використанням шаблону *Abstract Factory*, продемонстровано переваги інкапсуляції способів створення об'єктів і забезпечено розширюваність програмної системи без модифікації існуючого клієнтського коду.